ZEN + NoiFi

Basic implementations, small extensions, measured comparisons

Khaled Hasan Irfan | MacBook Air M5 | CPU | Python 3.10.22

Experiment	What was executed	Outcome
NoiFi basic	1103 real CSI recordings; compact CNN/DAE; FGSM + PGD	Clean test accuracy: 91.4%
NoiFi extension	N = 1, 3, 5, 10, 20; three permutation seeds	AUROC, recall, test FPR, latency and score SD measured
ZEN attribution	Upstream similarity code; supplied recovered artifacts	13/13 base families identified
ZEN patch demo	Local function bytecode; identical model weights; 64 inputs	Output error: 0.0 after patch
Full ZEN recovery	Supplied Linux memory dump + official YOLO patch inference	Not run: needs separate compatible setup and dump

Scope of the comparison

ZEN attributes and restores model computation. NoiFi detects adversarial CSI inputs. These solve different problems, so their accuracy and AUROC are not ranked against each other. Each is compared with an appropriate baseline or ablation.

What changed from the initial project

Installed Python 3.10; added a small runnable implementation, real-data download with hash checks, separate calibration/test partitions, fail-fast scripts, evidence CSVs, numerical output checking, and Python-generated plots/PDF. Removed completion claims based only on folders or logs.

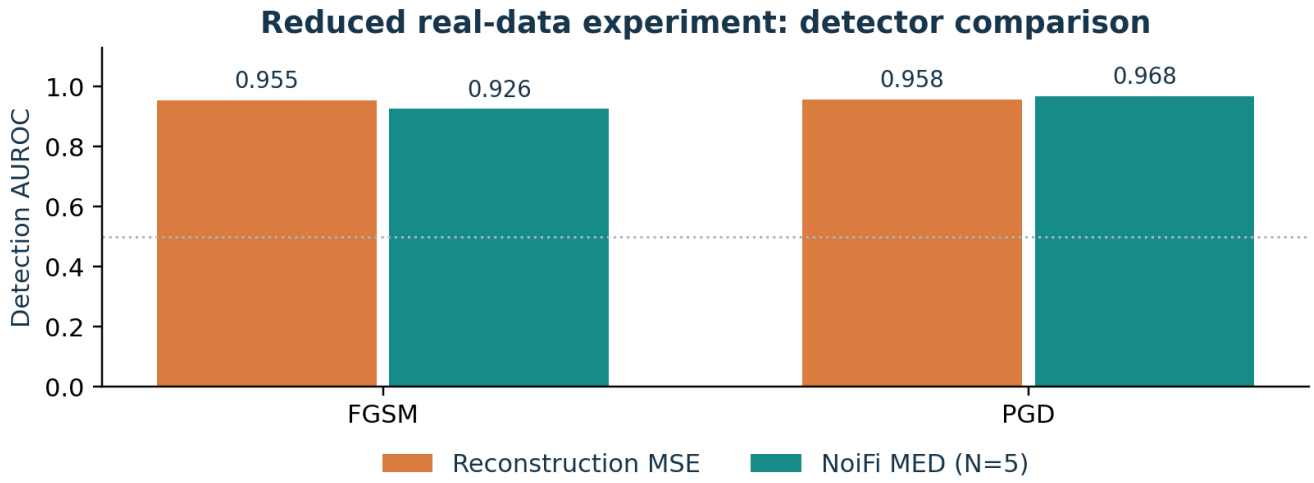
All reported values come from the CSV/JSON files in results/. No paper headline scores are presented as local results.

NoiFi | basic experiment

Real RoboFiSense CSI, eight classes. One sample per recording. Partitions: 661 train, 110 validation, 110 calibration, 222 final test. Inputs are normalized as in the upstream loader and resized to 64 x 64.

Mechanism: DAE reconstruction -> residual noise -> independent pixel permutations -> classifier penultimate features -> mean Euclidean distance (paper Eq. 8). The baseline uses reconstruction MSE without randomized features.

Clean test accuracy: 91.4%; uniform-noise control accuracy: 90.5%. The random control uses the same 0.10 amplitude bound; label preservation is not assumed.



Attack	ASR*	Detector	AUROC	Recall**	Test FPR
FGSM	96.1%	Reconstruction MSE	0.955	77.0%	5.9%
FGSM	96.1%	NoiFi MED N=5	0.926	59.5%	5.0%
PGD	100.0%	Reconstruction MSE	0.958	77.5%	5.9%
PGD	100.0%	NoiFi MED N=5	0.968	77.2%	5.0%

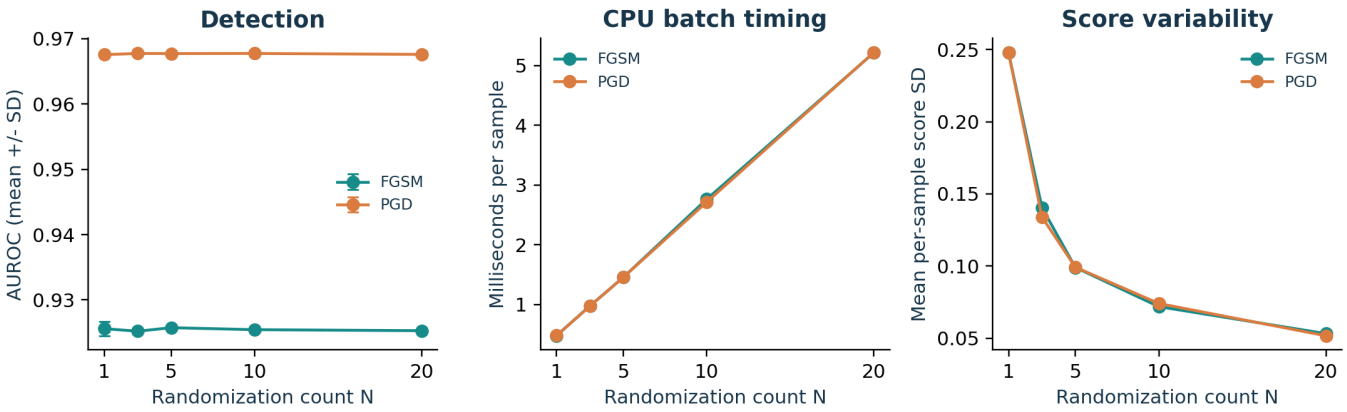
* ASR is measured only on clean-correct test recordings. Detection includes all test attacks. Both attacks use L-infinity epsilon = 0.10 on normalized inputs; PGD uses 20 steps and random start.

** Thresholds use a separate benign calibration partition with a 5% FPR target. The observed held-out test FPR is reported, not assumed to equal 5%. NoiFi rows average three permutation seeds; they share one trained model.

Observed comparison: FGSM AUROC is 0.955 for MSE and 0.926 for MED; PGD is 0.958 and 0.968, respectively. This reduced run does not show a universal improvement from MED. Small models and resized inputs do not establish the full paper results.

NoiFi | small extension

Question: how many randomized references are needed before scores stabilize, and what is the cost? The trained CNN, DAE, recordings and adversarial examples remain fixed. Only N and permutation seed vary.



N	Attack	AUROC mean +/- SD	Recall	Test FPR	ms/sample
1	FGSM	0.926 +/- 0.0011	59.8%	5.0%	0.48
1	PGD	0.968 +/- 0.00023	77.3%	5.0%	0.48
3	FGSM	0.925 +/- 0.0002	59.6%	5.0%	0.97
3	PGD	0.968 +/- 0.00014	77.5%	5.0%	0.97
5	FGSM	0.926 +/- 0.00033	59.5%	5.0%	1.45
5	PGD	0.968 +/- 0.00015	77.2%	5.0%	1.45
10	FGSM	0.925 +/- 0.00022	59.6%	5.0%	2.76
10	PGD	0.968 +/- 5e-05	77.2%	5.0%	2.71
20	FGSM	0.925 +/- 0.00027	59.5%	5.0%	5.21
20	PGD	0.968 +/- 6e-05	77.2%	5.0%	5.22

Timing includes DAE reconstruction, permutations, feature extraction and scoring. It is CPU wall time per sample amortized over batches of up to 32, not single-request latency. Initialization, model training and attack generation are excluded. These timings are hardware-specific.

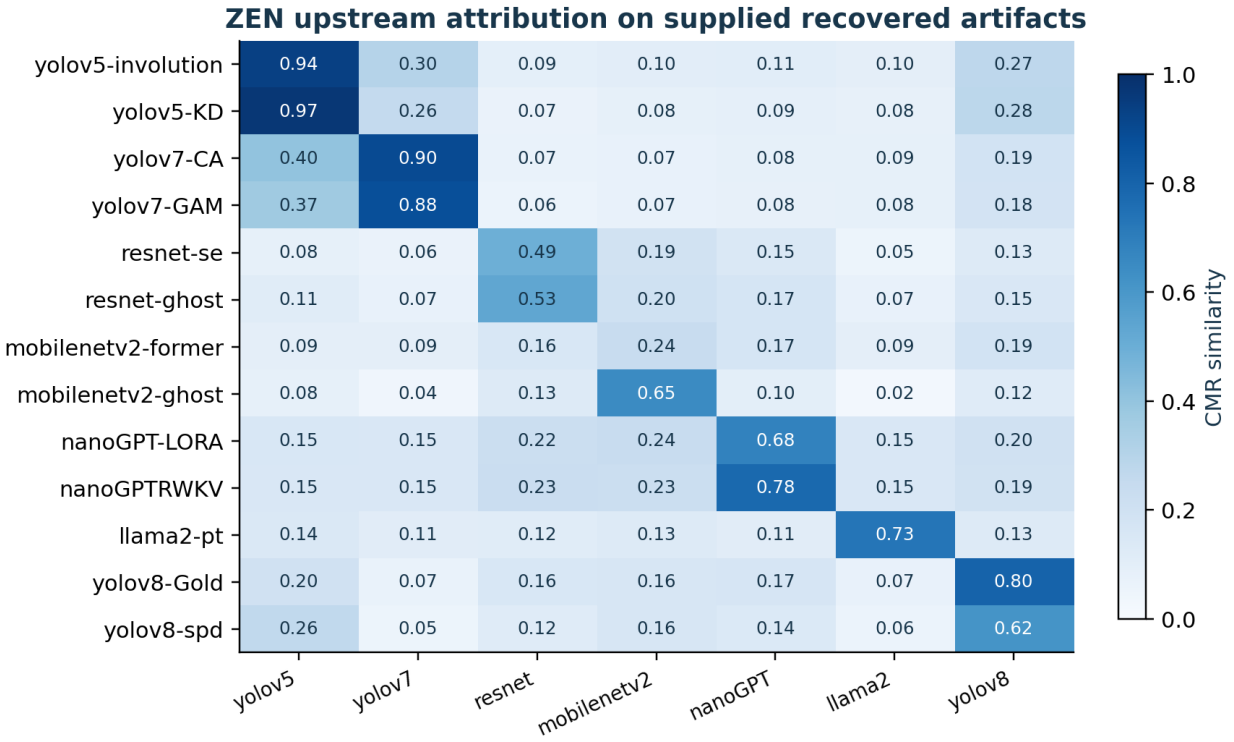
Score SD is first computed across the three permutation seeds for each held-out sample, then averaged across samples and benign/adversarial kinds. It measures randomization stability, not uncertainty across independently trained models.

Observed trade-off: AUROC changes very little across N. Score variability decreases, while per-sample CPU time grows from about 0.48 ms at N=1 to 5.22 ms at N=20. These measurements support using fewer references for this reduced setup; broader validation is needed.

ZEN | attribution and patching

Executed the upstream ZEN similarity algorithm on 13 authors-supplied recovered CMR artifacts against seven base candidates. Top-one family identification: 13/13. No new RAM image was recovered in this experiment.

Ablation comparison: mathematical representation alone (MR): 11/13; programmatic representation alone (PR): 13/13; combined representation (CMR): 13/13. This is a closed-set attribution check on the supplied artifacts.



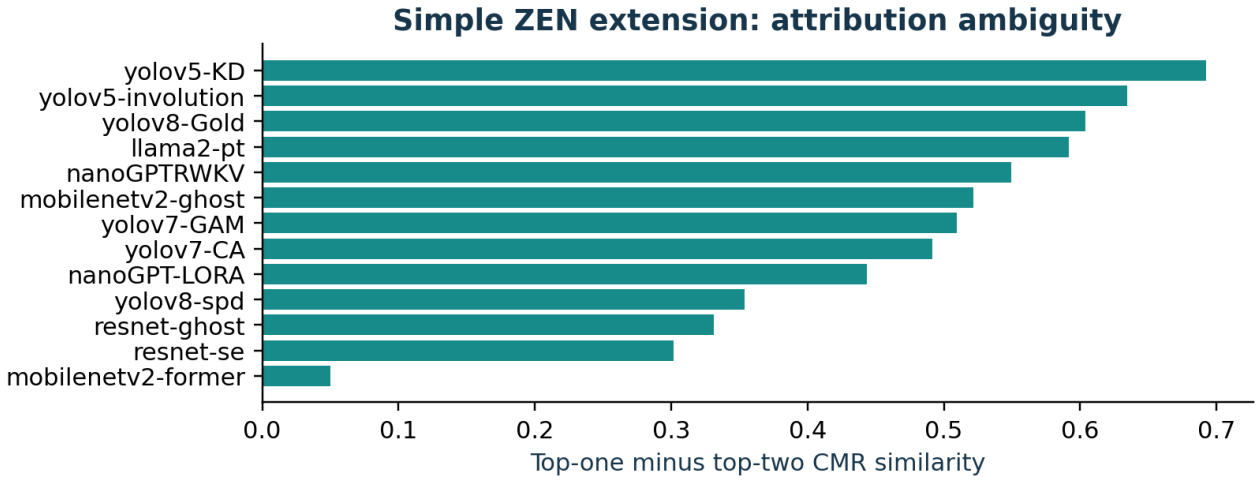
Artifact coverage: 1861 referenced bytecode files are absent across the 20 inspected models. Upstream missing-file behavior is retained and counts are in artifact_coverage.csv. This limits the interpretation of the similarity scores.

Controlled patch demonstration

With the same weights but the base forward function, maximum output error was 0.613492. Restoring our locally captured custom forward bytecode reduced it to 0.0 over 64 inputs. This illustrates code-plus-weights recovery; it is not the official YOLO memory experiment.

ZEN | extension and next steps

Simple extension: rank candidates and record the top-two similarity margin. A small margin flags cases worth inspecting; it is not a calibrated confidence score.



Smallest observed margin: mobilenetv2-former, 0.0498; predicted family mobilenetv2. All 13 supplied queries were still identified correctly in this closed candidate set.

What is still required

Full ZEN reproduction requires the supplied LiME dump, the matched Linux x86-64 / Python 3.8.18 environment, Volatility recovery, and successful official YOLO inference after patching. None of those steps is marked complete here. The original-vs-recovered output checker is ready for real tensor exports.

Sources and download links

ZEN official code: https://github.com/CyFI-Lab-Public/ZEN

NoiFi official code: https://github.com/shahriar0651/NoiFi

RoboFiSense recordings: https://github.com/SiamiLab/RoboFiSense

ZEN supplied memory dump: https://drive.google.com/file/d/1nsTN716aGoT7AU32HmuFQjCYYXmakjHR/view

Paper methods checked against the provided Achieving Zen and Adversarial Noise PDFs. Source commits are recorded in reports/upstream-commits.txt and results/noifi-basic/config.json.